D.O.M: Operationalising MLOps at the Edge

Shaun Walsh 20005831
Higher Diploma in Computer Science (Level 8) SETU

Supervisor: Peter Windle

Red Hat Emerging Technologies Team, Waterford

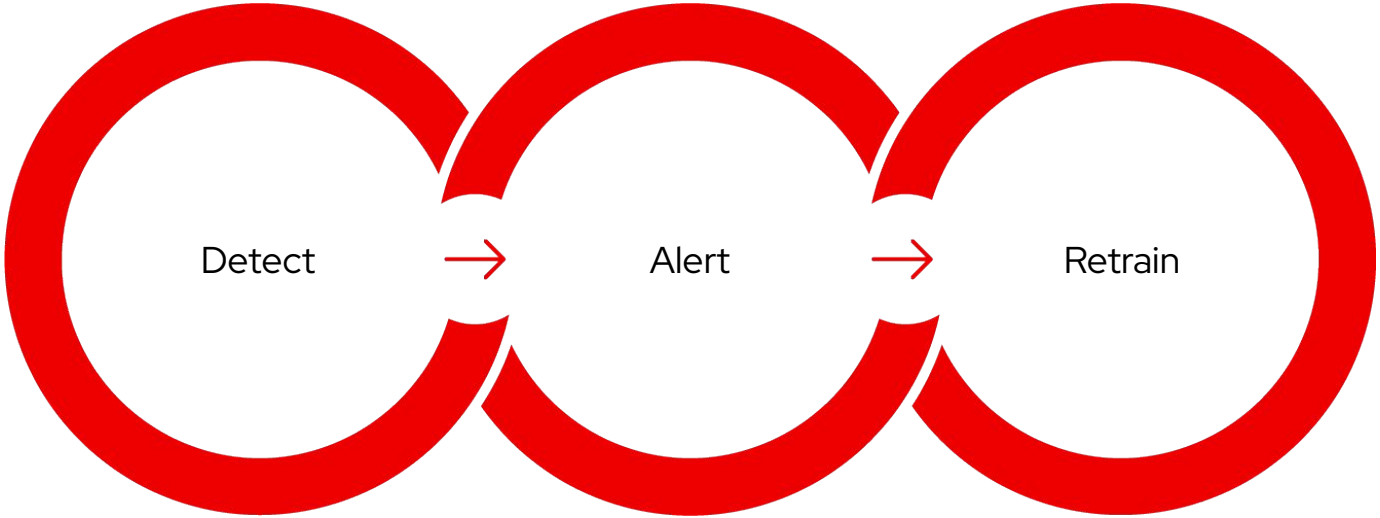
Red Hat

ML models deployed to edge devices degrade silently.

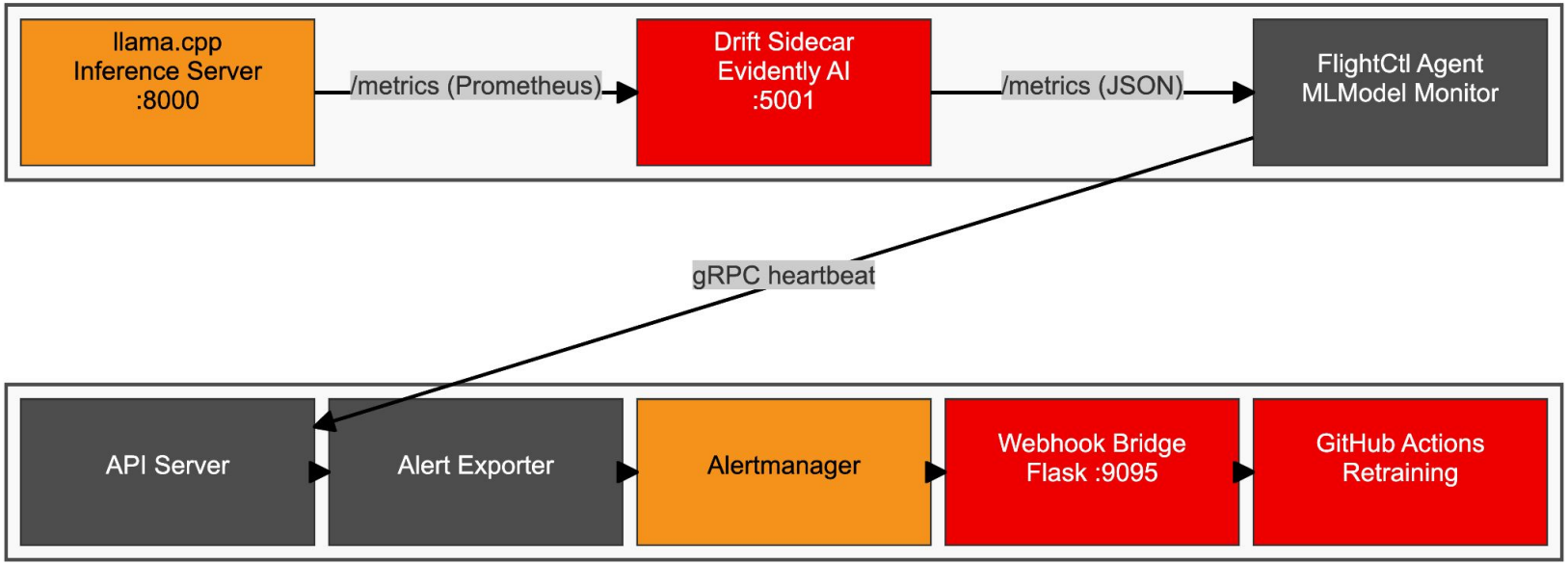
No monitoring. No alerts. Failures found only when users complain.

Automated Drift Detection to Retraining Trigger

Native ML monitoring capability added to FlightCtl



21 commits | 37 files | 2,337 lines | Go, Python, YAML



- New code
- Existing FlightCtl
- Third-party

Detection: The Sidecar Pattern

Swap the Sidecar not the Agent

Separation of Concerns

Agent stays a lightweight Go binary. ML logic stays in Python.

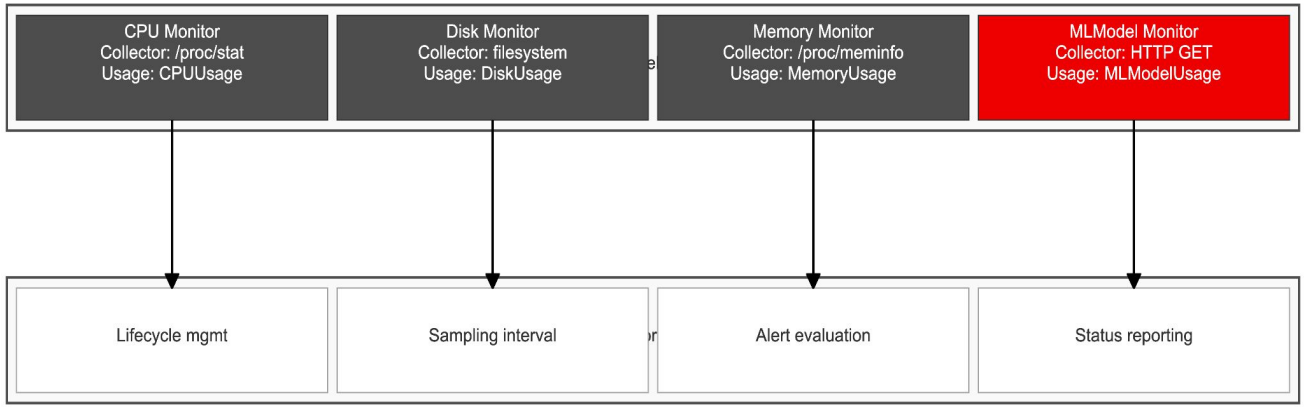
Framework agnostic

Any sidecar returning drift_score works.

Additive changes

Same Monitor/Collector pattern as CPU, Disk, Memory.

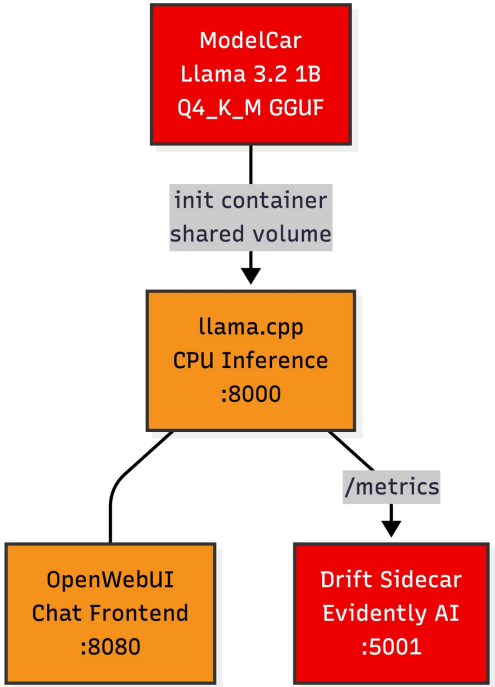
Reducing risk of regression



Following Upstream Patterns Exactly

- mlmodel.go: 287 lines (Monitor + Collector)
- mlmodel_test.go: 269 lines (3 test suites)
- convertDriftToPercentage(): bridges ML (0-1) to infra (0-100)
- One conversion function reuses the entire existing alert pipeline zero modifications

Reference MLOps Application



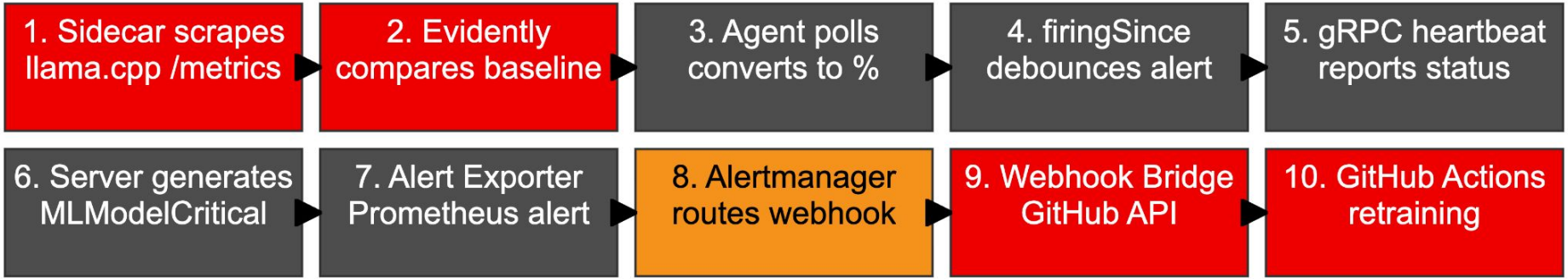
- ModelCar -- Llama 3.2 1B (Q4_K_M GGUF)
- llama.cpp -- CPU inference, Prometheus metrics
- Drift Sidecar -- Evidently AI (K-S test)
- Quadlets -- systemd-native deployment via FlightCtl

From detection to retraining trigger

Every component is real and working. Only the final retraining step is a placeholder.

Data Flow: From Metrics to Retraining

10-step path through the entire system



RAMP and Agile Kanban

Three pseudo-sprints tracked in JIRA Epic

Research (RAMP)
SPIKE sprint, 5 frameworks evaluated, MLOps courses

Sep-Oct 2025

MVP Test
Created MVP to test Evidently

Nov 2025

Sprint 1: Core Agent
API schema, monitor implementation, unit tests (6 commits)

Nov 2025

Sprint 2: Sample App & Device Setup
Sidecar, containers, BootC image, quadlets (9 commits)

Mar 2026

Sprint 3: Alert Pipeline
Webhook bridge, GitHub Actions, E2E test (6 commits)

Mar-Apr 2026

Key Design Decisions

Deliberate, Documented, and Defensible for PR

Decision	Rationale
Sidecar (not embedded)	Separation of concerns, framework agnostic
Operational drift (not data drift)	Pragmatic for edge; architecture supports both
llama.cpp (not vLLM)	CPU inference, native Prometheus, no GPU needed
Evidently AI (from 5 evaluated)	Lightweight, simple JSON output, edge-suitable
Pointer field for Mlmodel	Optional monitor vs mandatory CPU/Disk/Memory
convertDriftToPercentage()	One function reuses entire existing, alert pipeline with zero modifications
Quadlets (not Podman Compose)	Init container pattern for model delivery, systemd ordering guarantees weights are in the volume before inference starts

Problems Encountered and Solutions

Constraints & Engineering trade-offs.

- ARM64 Mac -> EC2 dev infrastructure
- vLLM too heavy -> Pivoted to llama.cpp + GGUF
- BootC image -> Manual AMI build via quay.io
- Data drift complexity -> Operational metrics + /simulate
- Cluster access lost -> Dummy pipeline proves architecture

What I Learned and Achieved

Optional subheading

Skills Gained

- Go systems programming & generics
- OpenAPI-first API design
- Container orchestration (Podman, Quadlets)
- MLOps pipeline architecture
- Edge computing infrastructure

Achievements

- First ML monitoring in FlightCtl
- Production-ready code for upstream PR
- Full-stack: API schema to CI/CD pipeline
- Patent application filed (USPTO)
- End-to-end validated on EC2

Future Development

The architecture was designed for these extensions

GUI Integration

Model alert and health info in FlightCtl frontend

OpenShift AI Integration

Real Kubeflow retraining pipeline/simulation in RHOAI

Fleet-Wide Analytics

Aggregate model health across entire fleet

Thank you

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make
Red Hat a trusted adviser to the Fortune 500.

linkedin.com/company/red-hat

youtube.com/@redhat

facebook.com/RedHat

x.com/RedHat

Red Hat